Animated texture interpolation ignores alpha channel during transition from/to transparent pixels

MC-144761 `Resource Packs`

Status	Resolved / Fixed
Affected	Minecraft 1.13.2, Minecraft 19w07a, Minecraft 19w08b, 1.16.3, 1.16.4 Release Candidate 1, 1.16.4 (+7 more)
Fixed in	24w33a
Reported	2019-02-22

STEPS TO REPRODUCE

1. Download and enable the attached `Interpolation test.zip` resource pack.
2. Create or enter a Creative world with default settings.
3. Open the Creative inventory and obtain an apple, or obtain a glass item affected by the resource pack.
4. Display the apple or glass texture so its animation is visible.
5. Wait a few seconds for the texture animation to transition between frames.
6. Observe that opaque pixels fading to transparent transition toward the transparent pixels' RGB color before becoming transparent abruptly, while transparent pixels fading to opaque remain invisible until the transition ends and then appear suddenly.

OBSERVED BEHAVIOUR

When an animated texture uses `"interpolate": true` and a pixel changes between opaque and transparent frames, interpolation does not correctly account for the alpha channel. When an opaque pixel transitions to a transparent pixel, the pixel is rendered using the transparent frame's RGB values while retaining visible opacity during the transition, then becomes fully transparent abruptly when the transition finishes. Consequently, a transparent pixel with RGB values of red, green, black, or any other color can produce that color during the fade even though its alpha value is zero. When a transparent pixel transitions to an opaque pixel, it remains invisible for the entire interpolation period and then suddenly appears with its final color when the next frame becomes active. This behavior occurs on both ordinary glass and stained glass, indicating that it is caused by the texture-frame interpolation process rather than by the affected block's support for partial transparency.

EXPECTED BEHAVIOUR

When interpolating animated textures, transparent pixels should be treated as transparent throughout the transition, regardless of their stored color values. Pixels fading to transparency should smoothly decrease in opacity without displaying unintended colors before becoming fully transparent, while pixels fading from transparency should gradually appear with their correct colors and opacity rather than remaining invisible until the transition completes.

ENVIRONMENT

Minecraft Java Edition Versions: 1.13.2, 19w07a, 19w08b, 1.16.3, 1.16.4 Release Candidate 1, 1.16.4, 1.16.5, 21w11a, 1.17.1, 1.18, 1.18.1, 1.18.2, 1.20.2 (includes snapshots)

ORIGINAL DESCRIPTION

The bug

If an animated texture that uses texture interpolation ({"interpolate": true} in the .mcmeta file) contains transparent pixels, the following behaviors can be observed:

- When fading TO transparent pixels, i.e. having a pixel that starts opaque and becomes transparent in the next frame, during the interpolation they will fade to whatever the RGB values for the pixels are, ignoring the alpha channel (the game just behaves as if the pixel was fully opaque). Once the transition is over, those pixels instantly change from that color to transparent, which creates a very jarring effect.

- When fading FROM transparent pixels, i.e. having a pixel that starts transparent and becomes opaque in the next frame, they'll appear completely transparent until the transition is over. Once it is over, the correct color will, again, appear all of a sudden.

Although this bug has no effect on the vanilla game, since no textures use interpolation and transparent pixels simultaneously, resource packs using both these features on the same texture should not encounter this issue.

Explanation

The example resource pack provided below (and shown above) is one I created to make a comparison between the old and new vanilla textures, which takes advantage of texture animations to cycle the textures every few seconds. Since many items have had their shape changed in the updated textures, many pixels that were transparent in an old texture are now "present" in the new texture, and vice-versa. As such, the effect described here became very apparent, which was what led me to uncover that bug in the first place.

The program I used to generate these textures happens to make every transparent pixel black (which is what most image editing software does), which at first made me think that the game would always use black as the color it transitioned towards before becoming transparent. After a few more tests, this has shown not to be the case.

Rather, any "color" that's stored in the RGB channels of transparent pixels will be used during the transition. For example, let's say I have a solid red pixel (255, 0, 0, 255) that fades to a transparent green pixel (0, 255, 0, 0). Noti

MANUAL RESULT

Version used	
Reproduced?	YES / NO / PARTIAL
Crash file	
Notes	
Tested by / date	

Live issue: https://bugs.mojang.com/browse/MC/issues/MC-144761